

Task 2 — Altitude PID Control, Explained

HACKSIMUL8 2026 · PES University · 2 September 2026 Status: COMPLETE — all numbers below are from an actual verified run

What Task 2 asked: "Design the Altitude Control System for 6-DOF UAV Quadcopter using PID Control."

Requirement: "Neat description of the methodology used for tuning of PID controller gains should be presented during the evaluation. Use the best method available for tuning."

Read that requirement carefully. **The tuning method is what is being graded, not the gains.** A team that drags sliders until the plot looks nice will lose to a team that can justify why they chose their method. This document is built around that.

1. The plant we are controlling

From Task 1, the altitude equation is:

$$\ddot{z} = -g + (U1/m) \cdot \cos \phi \cdot \cos \theta$$

Two things make this hard:

1. **It is nonlinear** — thrust is multiplied by $\cos \phi \cdot \cos \theta$. Tilt the quadcopter and you lose lift.
2. **It is a double integrator** — thrust produces acceleration, which integrates twice to position. Both poles sit at the origin.

Linearised about hover, $G(s) = 1/(m s^2) = 1/(0.516 s^2)$.

2. The key design move: feedback linearisation

Most teams will linearise about hover, design a PID for $1/(0.516 s^2)$, and accept that it degrades when the vehicle tilts. **We do something better.**

Instead of approximating the nonlinearity away, **cancel it exactly**. Command the thrust as:

$$U1 = m \cdot (g + u_z) / (\cos \phi \cdot \cos \theta)$$

Substitute back into the plant:

$$\begin{aligned} \ddot{z} &= -g + (1/m) \cdot [m(g + u_z) / (\cos \phi \cos \theta)] \cdot \cos \phi \cos \theta \\ &= -g + (g + u_z) \\ &= u_z \quad \leftarrow \text{exactly} \end{aligned}$$

The trig cancels **completely**. The PID now controls a clean $1/s^2$ — and this holds at **any** tilt angle, not just small ones.

What the two pieces do:

Term	Purpose
$m \cdot g$	gravity feed-forward — cancels weight so the PID starts from zero, not fighting gravity
$1/(\cos \phi \cos \theta)$	tilt compensation — restores exactly the lift lost to tilting

Proof that it works

In the Simulink model, altitude is commanded to 1 m, then at $t = 4$ s the vehicle is pitched to **0.3 rad (17°)**:

```
z at 3.9 s (before pitch) : 1.0000 m
z at 4.3 s (during pitch) : 1.0000 m
z at 6.0 s                : 1.0000 m
maximum sag from tilt    : 0.000002 m
```

Two microns of altitude error at 17° of tilt. A conventional hover-linearised controller would sag several centimetres. This is the single most convincing demonstration in Task 2 — show this plot.

3. Why Ziegler–Nichols cannot be used here

This is your strongest single line to a judge, because most teams will apply ZN mechanically and get nonsense.

The ultimate-gain (closed-loop) method requires finding a gain K_u at which the loop sustains a constant-amplitude oscillation. For a double integrator under proportional control:

$$m \cdot s^2 + K_p = 0 \quad \rightarrow \quad s = \pm j \cdot \sqrt{(K_p/m)}$$

The poles are **purely imaginary for every $K_p > 0$** . The system oscillates at *any* gain, so there is no unique ultimate gain to measure. **The method is undefined.**

The reaction-curve (open-loop) method requires an S-shaped step response from which to read a delay and a time constant. A double integrator's step response is a parabola that diverges — no S-curve, no reading.

Both classical Ziegler–Nichols variants are ill-posed for this plant. Saying this demonstrates you analysed the system rather than reaching for a recipe.

4. The four methods that do work

Presented as a **progression** — each step relaxes an assumption the previous one made.

Method A — Analytical pole placement

A PID acting on $\ddot{z} = u_z$ gives a **third-order** closed loop:

$$s^3 + K_d \cdot s^2 + K_p \cdot s + K_i = 0$$

A trap to avoid: do *not* design K_p and K_d as a PD and then bolt K_i on afterwards. Integral action moves the poles, destroying the damping you just designed for. (Doing exactly that in our first attempt produced 20% overshoot and no settling.)

Instead place **all three poles at once**. Choose a dominant complex pair plus one faster real pole at $s = -p$:

$$(s^2 + 2\zeta\omega_n s + \omega_n^2)(s + p) = s^3 + (2\zeta\omega_n + p)s^2 + (\omega_n^2 + 2\zeta\omega_n p)s + \omega_n^2 p$$

Matching coefficients gives closed-form gains:

$$K_d = 2\zeta\omega_n + p \qquad K_p = \omega_n^2 + 2\zeta\omega_n p \qquad K_i = \omega_n^2 \cdot p$$

Our specification: $\zeta = 0.9$, settling time $2\text{ s} \rightarrow \omega_n = 4/(\zeta \cdot T_s) = 2.222\text{ rad/s}$, and we take $p = 2\zeta\omega_n = 4.0$ so the real pole is twice as fast as the pair's envelope.

$$\begin{aligned} K_p &= 20.9383 & K_i &= 19.7531 & K_d &= 8.0000 \\ \text{closed-loop poles: } &-4, &-2 \pm 0.969i \\ \text{initial acceleration demand: } &20.9\text{ m/s}^2 &\text{vs } &29.4\text{ m/s}^2 \text{ available } &\checkmark \end{aligned}$$

That last check matters — we verify the design does not demand more acceleration than the rotors can physically produce.

Method B — `pidtune` (automatic, linear)

$$\begin{aligned} K_p &= 0.1857 & K_i &= 0.0087 & K_d &= 0.9739 \\ \text{crossover} &= 1.000\text{ rad/s} & \text{phase margin} &= 69.0^\circ \end{aligned}$$

MATLAB's automatic tuner. Note it chose a **very conservative** loop — crossover of only 1 rad/s. `pidtune` defaults to a cautious bandwidth for a marginally stable plant, so it gives a rock-solid 69° phase margin but a 6.5 s rise time. Safe, but sluggish.

Method C — `pidtune` with a target bandwidth

$$K_p = 1.1605 \quad K_i = 0.1358 \quad K_d = 2.4348 \quad (\omega_c = 2.5\text{ rad/s, PM} = 69.0^\circ)$$

Same tuner, but we specify the crossover frequency ourselves. Response time is roughly $1/\omega_c$. This shows you can trade speed against stability **deliberately** rather than accepting a default.

Method D — ITAE optimisation on the nonlinear model ← the best method

Directly minimise a performance index on the **full nonlinear simulation**, with actuator saturation and anti-windup active:

$$J = \text{ITAE} + 0.05 \cdot (\text{overshoot \%}) + 0.02 \cdot (\text{peak thrust above hover}) + 0.002 \cdot (\text{control variation})$$

Minimised with `fminsearch` (no Optimization Toolbox needed).

$$K_p = 32.8366 \quad K_i = 0.0000 \quad K_d = 10.4046$$

Why this is genuinely "the best method available": methods A, B and C all optimise a *linear approximation* of the plant. Method D optimises **what actually flies** — including the nonlinearity, the actuator limits, and the discrete 100 Hz controller.

ITAE (Integral of Time-weighted Absolute Error) is the right index for tracking because the time weighting punishes errors that *persist*, while tolerating the unavoidable error immediately after a step.

5. Results — the comparison table

All four evaluated on the identical nonlinear model with saturation:

Method	Kp	Ki	Kd	Tr [s]	Ts [s]	OS [%]	ITAE
A: Pole placement	20.938	19.753	8.000	0.420	2.830	26.21	0.5311
B: pidtune	0.186	0.009	0.974	6.480	—	0.00	11.1617
C: pidtune @ $\omega_c = 2.5$	1.160	0.136	2.435	2.590	—	15.88	5.0895
D: ITAE optimised	32.837	0.000	10.405	0.500	0.840	0.07	0.0696

Method D wins by 7.6× over the next best (0.0696 vs 0.5311), with 0.84 s settling and essentially zero overshoot.

— in the Ts column means the response never entered the $\pm 2\%$ band inside the simulation.

6. Two results you must be able to defend

Judges will notice both of these. Have the answers ready.

(a) Why did Method A overshoot 26% when it was designed for $\zeta = 0.9$?

Because pole placement controls poles, but a PID also creates zeros.

The closed-loop transfer function from reference to output has a numerator contributed by the controller. With integral action there is a zero at $s = -K_i/K_p \approx -0.94$. A **slow zero causes overshoot regardless of where you place the poles.**

Pole placement is blind to this — it only matches the characteristic (denominator) polynomial. Method D avoids the problem entirely because it optimises the *actual measured response*, in which zeros are automatically accounted for.

This is a genuine and instructive limitation of the analytical method, not a mistake.

(b) Why did Method D drive Ki to exactly zero?

Because after feedback linearisation there is nothing for integral action to do.

Gravity is already cancelled by the `m·g` feed-forward term, and the plant is exactly $1/s^2$. In the **nominal** case there is no steady-state error to remove, so integral action contributes nothing but overshoot — and the optimiser correctly deleted it.

But here is the important consequence: with `Ki = 0`, any *unmodelled* steady disturbance — a constant wind, or a mass error — produces a **permanent altitude error**. The nominal optimum is **not robust**.

This is a second, independent motivation for Task 3: the best fixed gains for the nominal case are the wrong gains the moment conditions change. It also generates a testable prediction — **the ML model should learn to reintroduce integral action when it detects wind**.

7. The engineering realism that earns credibility

A control demo without these is not a control demo. All are implemented in `sim_altitude.m` and in the Simulink model:

Feature	Why it matters
Actuator saturation	thrust clipped to <code>[0, 20.248]</code> N — the rotors' real limit
Clamping anti-windup	the integrator stops accumulating while saturated
Derivative on measurement	avoids the derivative "kick" on a step command
Discrete controller at 100 Hz	matches what real flight hardware runs
RK4 plant integration	accurate integration between control updates

Note on saturation in the results

Peak thrust reaches **20.248 N — exactly the saturation limit** — during the initial climb. This is **correct behaviour, not a bug**. Method D chose aggressive gains demanding 32.8 m/s^2 while the rotors deliver at most 29.4 m/s^2 , so the command clips briefly. The clamping anti-windup prevents the integrator winding up during that interval, which is why the response still shows only 0.07% overshoot.

If asked "*why does your thrust clip?*" — that is the answer.

8. The bridge to Task 3

The final figure in `task2_altitude_pid.m` runs the **same Method D gains** at masses $\times 1.0$, $\times 1.4$, $\times 1.8$ and $\times 2.2$ (simulating payload pickup). Performance visibly degrades as mass rises.

This plot is the argument for Task 3. Fixed gains, however well tuned, are tuned for *one* operating condition. Change the payload or add wind and they are no longer optimal. That is precisely the problem the problem statement describes:

"highly non-linear systems subjected to system parameters' variations and environmental disturbances mitigate the performance of PID controller... Online tuning of PID parameter is required."

Show the Task 2 result, then show it breaking, then introduce Task 3.

9. Files

File	Role
<code>task2_altitude_pid.m</code>	Runs all four methods, compares, saves <code>task2_pid.mat</code>
<code>sim_altitude.m</code>	Nonlinear closed-loop simulator — saturation, anti-windup, derivative on measurement, optional tilt/wind/noise
<code>perf_metrics.m</code>	Tr, Ts, OS, SSE, ITAE, IAE, ISE, peak and RMS control effort
<code>alt_cost.m</code>	The ITAE-based cost function minimised by Method D
<code>quad_altitude.slx</code>	Simulink implementation of the whole loop

10. How to present Task 2 (about 4 minutes)

1. **"The altitude plant is a double integrator, and it is nonlinear in tilt."** Show $\ddot{z} = -g + (U1/m)\cos \phi \cos \theta$.
2. **"So we feedback-linearise."** Show the substitution cancelling exactly to $\ddot{z} = u_z$. *"This is valid at any tilt, not just near hover."*
3. **"Ziegler–Nichols is undefined for this plant."** Explain why — no unique Ku, no S-curve. *This is your differentiator.*
4. **"We used four methods instead, in increasing order of realism."** Walk the table.
5. **"Method D is the best method available because it optimises the real system"** — nonlinear, saturated, discrete. 7.6× better ITAE.
6. **"And here is the proof the linearisation works."** Show the 17° pitch test: 2 microns of sag.
7. **"But fixed gains break when the payload changes."** Show the mass-sweep plot → hand over to Task 3.

Questions you will be asked

"Why is derivative action mandatory?" The plant has two poles at the origin. With P-only the closed-loop poles are purely imaginary — it oscillates forever at any gain. The derivative term supplies the damping.

"Why does your best controller have no integral term?" Because feedback linearisation already cancels gravity, so there is no steady-state error in the nominal case. Integral action would only add overshoot. But that also means it is not robust to wind — which is exactly what Task 3 addresses.

"What is ITAE and why that index?" Integral of Time-weighted Absolute Error. The time weighting penalises errors that persist while tolerating the unavoidable initial error after a step, which matches what we want from a tracking controller.

"Your thrust saturates — is that a problem?" No, it is realistic. The rotors have a finite limit and we model it. Clamping anti-windup handles the integrator during saturation, and the response still shows 0.07% overshoot.

"How do you know the gains are optimal?" They minimise a stated cost on the full nonlinear model. In Task 4 we go further and compare against an oracle that optimises per test condition, which bounds the best

achievable.